



Unifor

Counting Rooms

CSES 1192 — contando salas com busca em grafos

Alunos: Enzo Esmeraldo | Gustavo Andrade

Orientador: Prof. Me. Ricardo Carubbi

Disciplina: Resolução de Problemas com Grafos

Centro de Ciências Tecnológicas - CCT

Universidade de Fortaleza - UNIFOR



O problema

CSES 1192 — Counting Rooms

Mapa de $H \times W$ com paredes (#) e pisos (.)

Movimento só na horizontal e na vertical

Contar quantas regiões de piso existem

$1 \leq H, W \leq 1000 \rightarrow$ até 1.000.000 de posições

```
#####      #####  
#..#...#    #AA#BBB#  
####.#.#   →  ####B#B#  
#..#...#    #CC#BBB#  
#####      #####
```

Saída: 3





Modelagem

De mapa para grafo

Vértice: cada posição que contém “.”

Aresta: pisos vizinhos na horizontal ou vertical

Paredes (#) ficam fora do grafo

Não direcionado, não ponderado e desconexo

Contar salas = contar componentes conexos

	1	2	3	4	5	6	7	8
1	#	#	#	#	#	#	#	#
2	#	.	.	#	.	.	.	#
3	#	#	#	#	.	#	.	#
4	#	.	.	#	.	.	.	#
5	#	#	#	#	#	#	#	#

$|V| = 12$ $|E| = 10$
3 componentes





Representação

Grafo implícito na matriz

A própria matriz de caracteres é o grafo

Nenhuma aresta é armazenada

Os 4 vizinhos são calculados na hora

Aresta existe se: está dentro dos limites e contém “.”

Evita listas de adjacência para 10^6 vértices

```
int[] dLinha  = { -1, 1, 0, 0 };
int[] dColuna = {  0, 0, -1, 1 };
```

```
int vizLinha  = linha  + dLinha[direcao];
int vizColuna = coluna + dColuna[direcao];
```

```
if (vizLinha < 0 || vizLinha >= linhas ||
    vizColuna < 0 || vizColuna >= colunas)
    continue;
```

```
if (mapa[vizLinha][vizColuna] != '.' ||
    marcado[vizLinha][vizColuna]) continue;
```

E se fosse lista de adjacência?

Funcionaria igual: um `Bag<Integer>` por posição, vértice `id = linha * colunas + coluna`, cada piso ligado aos vizinhos piso — mesma BFS, mesma $O(H \cdot W)$.

Mas seriam ~4 milhões de entradas guardadas para responder o que a matriz já responde.





Busca

DFS no marco 3, BFS na submissão

Cada busca iniciada = uma sala (laço de algs4.CC)

DFS recursiva: empilha a região inteira

BFS: a fila guarda só a fronteira

mapa 1000×1000 aberto

DFS	pilha	1.000.000	StackOverflowError
BFS	fila	1.000	0,27 s

```
for (int linha = 0; linha < linhas; linha++)  
  for (int coluna = 0; coluna < colunas; coluna++)  
    if (mapa[linha][coluna] == '.'  
        && !marcado[linha][coluna]) {  
      bfs(linha, coluna);  
      salas++;  
    }
```

```
// bfs marca ao ENFILEIRAR,  
// entao ninguem entra 2x
```





Implementação da BFS

A função bfs: uma fila explícita sobre a matriz

Fila explícita no lugar da recursão da DFS

Marca o vizinho ao enfileirar, nunca ao remover

Vértice cabe em um int: linha $\times$ colunas + coluna

Fila vazia = uma sala inteira visitada

Sem recursão: nenhuma pilha para estourar

codificação do vértice

```
id      = linha * colunas + coluna
linha   = id / colunas
coluna  = id % colunas
```

```
private static void bfs(int linhaIni, int colunaIni) {
    Queue<Integer> fila = new Queue<Integer>();
    marcado[linhaIni][colunaIni] = true;
    fila.enqueue(linhaIni * colunas + colunaIni);

    while (!fila.isEmpty()) {
        int atual = fila.dequeue();
        int linha = atual / colunas;
        int coluna = atual % colunas;

        for (int direcao = 0; direcao < 4; direcao++) {
            // vizinho dentro do mapa, piso e nao marcado
            marcado[vizLinha][vizColuna] = true;
            fila.enqueue(vizLinha * colunas + vizColuna);
        }
    }
}
```





Validação e resultados

Testes, complexidade e submissão

6 casos pequenos + 2 de estresse: todos corretos

Diagonais (espera 4) pega erro de adjacência

Xadrez: 500.000 regiões em 0,26 s

Tempo e espaço: $O(H \cdot W)$

ACCEPTED no CSES — 19/19 testes, pior 0,34 s

caso	esper.	obtido	
exemplo oficial	3	3	
só paredes	0	0	
1×1 com piso	1	1	
3×3 aberto	1	1	
pisos em diagonal	4	4	
uma única linha	3	3	
1000×1000 aberto	1	1	0,27s
1000×1000 xadrez	500000	500000	0,26s





Unifor